2026年4月18日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年4月18日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル徹底比較　*特集（約4000字）*
3. EU AI Act 全面施行から半年——日本企業が今すぐ対応すべきコンプライアンス実務　*スタンダード（約2000字）*
4. AIエージェント時代の『ツール呼び出し』設計パターン5選　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

「コードを書く」という行為そのものが、2026年に入って決定的に変わりつつある。ターミナルに向かい、自然言語で意図を伝えれば、数百行のコードが生成され、テストが走り、プルリクエストまで作成される——そんなワークフローが、もはや一部のアーリーアダプターだけのものではなくなった。その中心にいるのが、Anthropicの**Claude Code**とOpenAIの**Codex CLI**だ。本コラムでは、2026年4月時点での最新動向を踏まえ、これらのAIコーディングエージェントを実務で最大限に活かすためのベストプラクティスを整理する。

エージェント型開発の現在地

2025年後半から2026年初頭にかけて、AIコーディングツールは「補完ツール」から「自律エージェント」へと明確にフェーズが移行した。Claude Codeは2025年のリリース以降急速に進化し、2026年初頭のアップデートでマルチファイル編集の精度とコンテキストウィンドウの実効的な活用効率が大幅に向上。一方のOpenAI Codex CLIもオープンソースとして公開され、ローカル環境でのサンドボックス実行やGitHub連携が強化されている。

両ツールに共通するのは、「人間が設計意図を伝え、AIがコードの実装・テスト・修正を自律的に回す」というエージェンティックなループが実用レベルに達したという点だ。では、このパラダイムでどう立ち回るべきか。

ベストプラクティス1：CLAUDE.mdとシステムプロンプトによるプロジェクト文脈の明示化

Claude Codeを使う上で最も効果を実感できるのが、リポジトリルートに配置する`CLAUDE.md`ファイルだ。ここにプロジェクトのアーキテクチャ方針、コーディング規約、使用フレームワーク、テストの実行コマンド、禁止パターンなどを記述しておくと、エージェントがセッション開始時にこれを読み込み、プロジェクト固有の文脈を踏まえたコード生成を行う。

2026年3月のアップデートでは、`CLAUDE.md`のネスト構造がサポートされ、ディレクトリ単位で異なる指示を与えられるようになった。たとえば`/backend/CLAUDE.md`にはAPI設計のルールを、`/frontend/CLAUDE.md`にはコンポーネント設計の方針を書く、といった運用が可能だ。Codex CLIでも`codex.md`や`AGENTS.md`による同様のアプローチが有効であり、**プロジェクトの暗黙知をドキュメント化する習慣**がAIエージェント時代の必須スキルになっている。

ベストプラクティス2：タスク分割と段階的な指示設計

エージェントに「このアプリを作って」と丸投げしても、期待どおりの結果は得られない。効果的なのは、**人間がアーキテクチャレベルの分割を行い、各タスクを明確なゴールとともにエージェントに渡す**アプローチだ。

具体的には、以下のような段階を踏む：

1. **設計フェーズ**：エージェントに要件を伝え、設計案を複数提示させる。人間がレビューし方針を決定する。
2. **実装フェーズ**：モジュール単位でタスクを切り出し、1タスク1セッションで実行する。
3. **検証フェーズ**：テストの生成と実行をエージェントに任せ、結果を人間が確認する。

Claude Codeの`/compact`コマンドによるコンテキスト圧縮や、Codex CLIの`--full-auto`モードを活用すれば、長時間のセッションでもコンテキストの劣化を

抑えられる。ただし、**1セッションで扱うスコープは意図的に小さく保つ**のが鉄則だ。

ベストプラクティス3：テスト駆動とサンドボックスの徹底活用

AIが生成したコードを本番に入れる以上、テストの重要性はむしろ増している。ベストプラクティスとして定着しつつあるのが、**先にテストをエージェントに書かせ、その後に実装を生成させる**TDD的アプローチだ。

Claude Codeでは`-p`（パイプモード）でCI/CDパイプラインに組み込むことが可能になり、プルリクエストの自動生成からテスト実行、失敗時の自動修正までを一連のフローとして構築できる。Codex CLIも同様に、ネットワーク隔離されたサンドボックス内でコマンドを実行する設計になっており、安全性を担保しながら自律実行が可能だ。

重要なのは、**エージェントが生成したテストそのものもレビュー対象にする**ことだ。カバレッジの数値だけを見て安心するのではなく、境界値やエラーケースが適切に網羅されているかを人間の目で確認する工程を省略してはならない。

ベストプラクティス4：並列エージェント実行によるスループット最大化

2026年に入り注目されているのが、**複数のエージェントセッションを並列で走らせる**ワークフローだ。Claude Codeでは複数のターミナルウィンドウ、あるいはヘッドレスモードで同時にタスクを実行し、それぞれが独立したブランチで作業を進める。Codex CLIでも同様に、複数のIssueに対して並列にエージェントを割り当てる運用が広がっている。

この手法では、**ブランチ戦略とマージポリシーの設計**が成功の鍵を握る。各エージェントが触るファイルのスコープを事前に分離し、コンフリクトの発生を最小化する設計力が求められる。結局のところ、AIエージェントの生産性を最大化するのは、人間のアーキテクチャ設計能力なのだ。

人間の役割はどこへ向かうのか

Claude CodeもCodex CLIも、驚くべき速度で進化を続けている。しかし、2026年4月現在、「AIがすべてを置き換える」という段階にはまだない。設計判断、セキュリティレビュー、ビジネスロジックの妥当性検証——これらは依然として人間の領域だ。

最も生産性が高いのは、**AIを"ジュニアエンジニアのチーム"として扱い、人間がテックリードとして全体を統括する**モデルである。指示の明確さ、文脈の整備、レビューの質——これらの「メタスキル」こそが、AIエージェント時代のエンジニアに求められる核心的な能力だ。ツールは変わる。しかし、優れたソフトウェアを作るための思考の本質は変わらない。

GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル徹底比較

2026年の桜が散り始めた今、AI業界の勢力図もまた大きく塗り替わろうとしている。OpenAIの**GPT-5**、Googleの**Gemini 2.5 Pro/Ultra**、そしてAnthropicの**Claude 4（Opus / Sonnet）**——この春、三大フロンティアモデルがついに出揃った。どのモデルも「史上最高」を謳い、ベンチマークの数字は軒並み人間の専門家レベルを超えてきている。だが、現場のエンジニアが本当に知りたいのは「結局どれを使えばいいのか」というただ一点だろう。本稿では、2026年4月時点で利用可能な最新バージョンを対象に、性能・コスト・マルチモーダル対応・推論能力・安全性を横断的に比較し、実務でのモデル選定に役立つ判断軸を提供する。

2026年春、三つ巴の構図はどう変わったか

2025年後半から2026年初頭にかけて、フロンティアモデルのリリースラッシュは加速の一途をたどった。OpenAIは2025年末にGPT-5を段階的にロールアウトし、2026年2月にAPI一般提供を開始。Googleは2025年12月にGemini 2.5 Proを公開、続いて2026年3月にGemini 2.5 Ultraをリリースした。Anthropicは2026年1月にClaude 4 Sonnetを先行公開し、3月にフラッグシップのClaude 4 Opusを投入している。

興味深いのは、三社がそれぞれ異なる設計哲学を鮮明にしてきたことだ。OpenAIは「汎用性と開発者エコシステム」、Googleは「マルチモーダルとスケール」、Anthropicは「推論の深さと安全性」を前面に打ち出している。かつてのように単一のベンチマークスコアで優劣が決まる時代は完全に終わり、**ユースケースに応じた最適解が異なる**という状況が定着した。

ベンチマーク比較：数字で見るフロンティアの現在地

以下に、2026年4月時点で公開されている主要ベンチマークの比較をまとめる。なお、数値は各社公式テクニカルレポートおよびサードパーティの再現実験（LMSYS Chatbot Arena、HELM 2026 Q1等）に基づくが、評価条件の違いには注意が必要だ。

ベンチマーク	GPT-5	Gemini 2.5 Ultra	Claude 4 Opus
MMLU-Pro（知識・推論）	92.4%	91.8%	93.1%
GPQA Diamond（大学院レベル科学）	74.9%	76.3%	78.2%
HumanEval+（コード生成）	95.1%	93.7%	96.3%

ベンチマーク	GPT-5	Gemini 2.5 Ultra	Claude 4 Opus
SWE-bench Verified（実プロジェクトバグ修正）	62.8%	58.4%	67.1%
MATH-500（数学推論）	98.2%	97.6%	97.9%
MathVista（視覚＋数学）	78.4%	82.1%	75.9%
MMMU（マルチモーダル理解）	75.8%	79.3%	73.2%
Chatbot Arena ELO（2026年3月）	1387	1379	1391

総合力ではGPT-5とClaude 4 Opusが僅差でリードし、特にClaude 4 Opusはコーディングタスクと GPQAで頭一つ抜けている。一方、**マルチモーダル系のベンチマークではGemini 2.5 Ultraが明確に強い**。MathVistaやMMMUでの優位性は、Googleがネイティブマルチモーダルアーキテクチャに投資してきた成果と言えるだろう。

ただし、ベンチマーク至上主義には警鐘を鳴らしておきたい。LMSYS Chatbot ArenaのELOスコアは人間の主観評価を反映するが、上位モデル間の差はもはや統計的有意とは言い難い水準に収束している。**実務での選定はベンチマーク以外の要因——コスト、レイテンシ、コンテキスト長、エコシステム——で決まることが多い。**

コスト比較：トークン単価と実効コストの罠

2026年4月時点のAPI価格（入力/出力、100万トークンあたり）を比較する。

モデル	入力単価	出力単価	コンテキスト長
GPT-5	$8	$24	256K
GPT-5 Mini	$1.5	$6	128K
Gemini 2.5 Ultra	$7	$21	2M
Gemini 2.5 Pro	$1.25	$5	1M
Claude 4 Opus	$15	$75	1M
Claude 4 Sonnet	$3	$15	200K

一見してわかるのは、**Claude 4 Opusの出力単価が突出して高い**ことだ。これはAnthropicが「拡張思考（Extended Thinking）」を標準で有効化しており、内部推論トークンが出力にカウントされるためでもある。推論トークンの割引適用後でも他社フラッグシップの2〜3倍のコストになるケースがある。

一方で**Gemini 2.5のコンテキスト長は圧倒的**だ。2Mトークンのコンテキストウィンドウは、大規模コードベース全体を一括投入するようなユースケースで他に代えがたい価値を持つ。さらにGoogleはBigQueryやVertex AIとの統合でバッチ推論の割引を積極的に提供しており、大量処理時の実効コストはカタログ価格よりかなり下がる。

コスト最適化の実践的なTipsとしては、以下のパターンが有効だ：

1. **ルーティング戦略**：簡易な質問はGPT-5 MiniやGemini 2.5 Proで処理し、複雑な推論タスクのみClaude 4 OpusやGPT-5にエスカレーションする。OpenAIが提供するモデルルーターAPIを活用すれば、このハイブリッド構成を比較的少ない工数で実装できる。
2. **プロンプトキャッシュ**：三社ともプロンプトキャッシュ機能を提供しており、繰り返し同じシステムプロンプトを使う場合は入力コストを50〜90%削

減可能。特にAnthropicのキャッシュは1Mトークンのコンテキストでも有効で、RAGパイプラインとの相性がよい。

3. **バッチAPI**：リアルタイム応答が不要なら、各社のバッチ推論エンドポイントで最大50%のコスト削減が見込める。

マルチモーダル対応：画像・音声・動画の処理能力

2026年春時点で、マルチモーダル対応の成熟度には明確な差がある。

Gemini 2.5 Ultraはこの領域のリーダーだ。テキスト・画像・音声・動画をネイティブに処理し、最大2時間の動画を直接入力できる。音声の感情認識やリアルタイムストリーミング入力にも対応しており、Google Meetとの統合で議事録生成＋感情分析を同時にこなすデモは記憶に新しい。

GPT-5は画像・音声の入出力に対応し、GPT-4oで培ったリアルタイム音声会話機能をさらに強化した。特筆すべきは画像生成能力で、モデル内部でネイティブに高品質な画像を生成できるようになった。DALL-E統合ではなくモデル自身が画像を「考えて描く」ため、テキストと画像の整合性が飛躍的に向上している。

Claude 4 Opusは画像入力とPDF解析に強いが、音声や動画の直接入力は2026年4月時点ではまだベータ段階だ。ただし、構造化されたドキュメント（技術仕様書、論文、法務文書など）の理解精度では最高峰であり、テキストヘビーな業務ではこの強みが光る。

マルチモーダルの選定指針をまとめると：

- **動画・音声中心のアプリケーション** → Gemini 2.5 Ultra一択
- **画像生成を含むクリエイティブ用途** → GPT-5が有力
- **文書理解・分析が主体** → Claude 4 OpusまたはGPT-5

推論能力：思考するモデルの真価

2026年のフロンティアモデルを語るうえで避けて通れないのが「推論（Reasoning）」能力の進化だ。各社とも、Chain-of-Thought（CoT）を内部化した「思考モデル」を標準機能として統合してきた。

Claude 4 Opusの拡張思考は、最大128Kトークンの内部推論チェーンを展開でき、複雑な数学証明やコードアーキテクチャ設計で他社を凌駕する。SWE-bench Verifiedで67.1%という数字は、実際のGitHubイシューの3分の2をモデル単体で解決できることを意味し、これはソフトウェアエンジニアリングの実務に直結するインパクトだ。

GPT-5はo3系列の推論技術を統合し、「推論の深さ」をAPIパラメータで制御できる。low / medium / highの三段階で推論コストと精度のトレードオフを明示的に管理できるのは、プロダクション環境では極めて実用的だ。

Gemini 2.5 UltraのThinking Modeは、特にマルチモーダル入力を含む推論で真価を発揮する。グラフや図表を読み解きながら論理的に結論を導く能力は、金融レポート分析や医療画像診断の補助など、視覚情報と論理推論の統合が求められる場面で強みを持つ。

安全性・ガバナンス：エンタープライズ導入の鍵

エンタープライズでの導入を考えるなら、モデル性能だけでなく安全性とガバナンスの体制も評価軸に加えるべきだ。

Anthropicは安全性で業界をリードし続けている。 Claude 4では「Constitutional AI 2.0」と呼ばれる改良版アラインメント手法を採用し、有害出力率を前世代比で60%削減したとされる。また、モデルの判断過程をある程度可視化する「解釈可能性ダッシュボード」をAPI利用者に提供しており、規制対応が求められる金融・医療領域で高い支持を得ている。

OpenAIはGPT-5で「Instruction Hierarchy」を強化し、プロンプトインジェクション耐性を大幅に向上させた。エンタープライズ向けにはデータ非学習保証やSOC 2 Type II認証をはじめ、EU AI Act対応のコンプライアンスパッケージも用意している。

GoogleはVertex AI上での展開を軸に、VPC Service Controls、Customer Managed Encryption Keys（CMEK）、データリージョン指定といったクラウドネイティブなガバナンス機能で差別化している。既にGoogle Cloudを利用している企業にとっては、追加のセキュリティ投資を最小化できる点が大きい。

実務で使えるモデル選定フローチャート

最後に、現場のエンジニアがモデルを選定する際の判断フローを提案する。

ステップ1：タスクの性質を特定する - コーディング・複雑な推論が主体 → Claude 4 Opus / Sonnet を第一候補 - マルチモーダル（画像・動画・音声）が必須 → Gemini 2.5 Ultra / Pro を第一候補 - 汎用的なテキスト生成・チャット → GPT-5 / GPT-5 Mini を第一候補

ステップ2：コスト制約を評価する - 月間の推論コスト上限がシビア → 各社の軽量モデル（GPT-5 Mini、Gemini 2.5 Pro、Claude 4 Sonnet）を検討 - 品質最優先で予算に余裕あり → フラッグシップモデルを使い、ルーティングで最適化

ステップ3：非機能要件を確認する - データレジデンシー要件 → Google Cloud / Azure上のデプロイオプションを確認 - レイテンシ要件が厳しい → ストリーミング応答のfirst token latencyを実測比較 - 長大なコンテキストが必要 → Gemini 2.5の2Mコンテキストが唯一の選択肢になる場合も

ステップ4：プロトタイプで実測する - ベンチマークはあくまで参考値。自社の実データで精度・レイテンシ・コストを実測し、A/Bテストで最終判断する。LMSYS風の盲検比較を社内で回すのも有効だ。

まとめ：最適解は一つではない

2026年春のフロンティアモデルは、いずれも驚異的な能力を備えている。しかし「万能の最強モデル」は存在しない。Claude 4 Opusのコーディング能力に惚れ込んでも、コストが見合わなければ採用できない。Gemini 2.5 Ultraの2Mコンテキストに魅力を感じても、自社のタスクがテキスト中心なら持て余すだろう。

現実解は「マルチモデル戦略」にある。 タスクの性質に応じてモデルを使い分け、ルーティングレイヤーで最適化する。そしてモデルのバージョンアップサイクルは加速する一方だから、特定モデルへのロックインを避け、抽象化レイヤーを挟んだアーキテクチャにしておくことが、2026年以降も変わらない鉄則だ。

フロンティアモデルの競争は、私たちエンジニアにとっては歓迎すべき状況だ。競争が激しいからこそ性能は上がり、価格は下がり、選択肢は広がる。この恵まれた環境を最大限に活かすために、ベンチマークの数字に踊らされず、自分のユースケースに正直に向き合おう。最適なモデルは、あなたの課題の中にしかない。

EU AI Act 全面施行から半年——日本企業が今すぐ対応すべきコンプライアンス実務

「うちはEU域内に拠点がないから関係ない」——もし今なおそう考えているなら、その認識は危険だ。2025年8月2日にEU AI Actの高リスクAI規制が本格適用されてから約8か月。すでに欧州市場向けにSaaSを提供していた日本のHRテック企業が、EU加盟国の市場監視当局から是正勧告を受けた事例が報じられてい

る（2026年2月）。GDPRの「域外適用」で痛い目を見た企業は少なくないが、AI Actでもまったく同じ構図が繰り返されつつある。本コラムでは、施行半年の運用実態を踏まえ、日本企業が"今すぐ"着手すべきコンプライアンス実務を具体的に整理する。

EU AI Actの域外適用——なぜ日本企業が対象になるのか

AI Act第2条は、**EU域内で利用されるAIシステムの提供者（provider）および配備者（deployer）** に対して、拠点の所在地を問わず義務を課す。つまり、日本国内で開発・運用するAIであっても、そのアウトプットがEU域内の自然人に影響を及ぼす場合は規制対象となる。典型例は以下の通りだ。

- EU企業にOEM供給している採用スクリーニングAI
- EU顧客にサービス提供しているAIベースの信用スコアリング
- 欧州子会社が利用している社内向けAIチャットボットや人事評価モデル

2026年2月に欧州委員会が公表した**最初の執行ガイダンス（Enforcement Guidance v1.0）** では、「域外事業者にも同等の技術文書・適合性評価を求める」方針が明確化された。罰則は最大3,500万ユーロまたは全世界売上高の7%（いずれか高い方）であり、GDPRを上回る規模感である。

高リスクAI分類と日本企業への影響マップ

AI Actの附属書IIIが定める「高リスクAI」カテゴリのうち、日本企業が特に注意すべき領域を挙げる。

1. **雇用・労務管理**：採用選考、業績評価、解雇判断に用いるAI
2. **信用・金融**：ローン審査、保険引受、信用スコアリング
3. **教育**：入学選考、学習評価のスコアリング
4. **重要インフラ**：交通・エネルギー分野の安全コンポーネント
5. **法執行・入国管理**：リモート生体認証（リアルタイム型は原則禁止）

2026年3月に欧州AI局（AI Office）が公開した**業種別FAQ第2版**では、SaaS型で提供するAIモデルについても「提供者責任」が発生し得ることが改めて確認された。汎用目的AIモデル（GPAI）に関するCode of Practiceの最終版も2025年末に確定しており、基盤モデル提供者には透明性義務が課される点も見逃せない。

対応チェックリスト——7つの実務ステップ

以下は、筆者がEU規制当局のガイダンスおよび国内法律事務所のアドバイザリーを総合して整理した実務チェックリストである。

1. **AIシステム棚卸し**：自社が開発・利用するAI資産を一覧化し、EU域内への影響有無を判定する。
2. **リスク分類マッピング**：附属書IIIに照らし、高リスク・限定リスク・最小リスクに分類する。
3. **適合性評価プロセスの設計**：高リスクAIには、リスク管理システム（第9条）、データガバナンス（第10条）、技術文書（第11条）、ログ保持（第12条）、透明性（第13条）、人間による監視（第14条）、正確性・堅牢性・セキュリティ（第15条）への適合が求められる。
4. **EU域内の認定代理人（Authorised Representative）の選任**：域外事業者は書面による委任が必須。
5. **EU AIデータベースへの登録**：高リスクAIは欧州委員会のデータベースに登録義務がある。
6. **事後モニタリング体制**：市場投入後も継続的な監視と重大インシデントの報告義務（72時間以内）を遵守する。
7. **社内教育・AIリテラシー義務**：第4条に基づき、AIを扱う従業員へのリテラシー教育が全リスクレベルで求められている。

国内ガイドラインとの差分——ギャップはどこにあるか

日本では、総務省・経済産業省が策定した「**AI事業者ガイドライン**」（**2024年4月公表、2025年12月改訂版**） が事実上の国内標準となっている。しかし、EU AI Actとの間には構造的なギャップが存在する。

項目	EU AI Act	日本AIガイドライン
法的拘束力	**法規制（罰則あり）**	ソフトロー（自主的遵守）
リスク分類	4段階の法定分類	原則ベースで分類は事業者判断
適合性評価	第三者評価または自己評価（類型による）	自己チェックが中心
技術文書義務	詳細なフォーマットを指定	記載推奨にとどまる
域外適用	明文規定あり	規定なし
汎用目的AIモデル規制	GPAI固有の透明性・評価義務	基盤モデル固有の規定は限定的

このギャップを埋めるために、日本企業は国内ガイドラインへの準拠を「ベースライン」、EU AI Actへの対応を「アドオン」として二層構造で管理するアプローチが現実的だ。2026年3月の自民党AI戦略PTの提言でも、国内法制化の検討が加速する見通しが示されており、今のうちにEU水準の体制を敷いておけば、将来の国内規制にも先回りできる。

まとめ——「様子見」はもう間に合わない

EU AI Actの施行は「遠い海の向こうの話」ではなく、EU域内にユーザーやパートナーを持つすべての日本企業にとって現在進行形のリスクである。2026年8月には汎用目的AIモデルへの完全適用も控えており、規制の波は広がる一方だ。まずはAI資産の棚卸しと高リスク分類の判定から始めてほしい。最初の一歩は地味

だが、その一歩を踏み出した企業と踏み出さなかった企業の差は、半年後には取り返しのつかないものになっているはずだ。

AIエージェント時代の『ツール呼び出し』設計パターン5選

2026年に入り、AIエージェントの「ツール呼び出し」はもはや実験的機能ではなくなった。MCPの事実上の標準化、OpenAIのAgents SDK v2、GoogleのAgent Development Kit（ADK）正式版——各社のフレームワークが出揃った今、外部ツール連携の設計パターンは急速に収束しつつある。現場で頻出する5パターンを、コード片とともに一気に紹介する。

1. Single-Shot Tool Call（単発呼び出し）

最も基本のパターン。LLMが1回の推論で1つのツールを選択・実行し、結果を返す。

```
@tool
def get_weather(city: str) -> str:
    return fetch_api(f"/weather?city={city}")
```

MCP対応サーバーでも `tools/call` エンドポイント1本で完結する。シンプルだが、複合タスクには力不足。

2. Sequential Chain（逐次チェーン）

ツールAの出力をツールBの入力に渡す直列パイプライン。ReActスタイルのループで自然に実現でき、OpenAI Agents SDKの `handoff` 機構とも相性が良い。依存関係が明示的でデバッグしやすい反面、レイテンシが直列に積み上がる。

3. Parallel Fan-Out（並列展開）

独立した複数ツールを同時呼び出しする。OpenAIの `parallel_tool_calls` やADKの並列ステップがこれに該当する。

```
# OpenAI Agents SDK v2 スタイル
response = await agent.run(
    tools=[get_weather, get_news, get_stock],
    parallel_tool_calls=True
)
```

レイテンシ削減に直結するため、本番では最も費用対効果が高いパターンだ。

4. Human-in-the-Loop Gate（承認ゲート）

高リスク操作（決済・データ削除など）の前に人間の承認を挟む。MCPでは `sampling` メッセージにconfirmationフローを組み込む設計が推奨されている。Agents SDKの `guardrails` 機能と組み合わせれば、ポリシー違反を実行前にブロックできる。

5. Dynamic Tool Discovery（動的ツール探索）

事前にツール一覧をハードコードせず、MCPサーバーの `tools/list` を実行時に叩いてスキーマを取得するパターン。プラグイン数が増え続ける環境で威力を発揮する。

```
available = await mcp_client.list_tools()   # 動的に取得
agent = Agent(tools=available)              # その場で注入
```

Google ADKの「Extension」機構も同様の思想で、ツールレジストリからの動的ロードを前提としている。

まとめ——パターンを"組み合わせる"時代へ

実務では、これら5パターンは排他的ではなく**組み合わせて**使う。たとえば「Dynamic Discoveryで取得したツール群をParallel Fan-Outし、破壊的操作だけHuman-in-the-Loop Gateを通す」といった構成が現実解になる。

フレームワークの乱立は続いているが、設計パターンのレイヤーで見れば共通言語が生まれつつある。まずはこの5つを手元のプロジェクトに当てはめてみてほしい。驚くほど設計判断がクリアになるはずだ。